Project

Retail Business Intelligence & Profitability Analysis Using Python

1. Project Introduction

The purpose of this project is to analyze retail sales data and find useful information about sales, profit, orders, customers, product categories, regions and monthly performance. Python is used to load, clean, inspect, calculate and visualize the data. The work is completed in Google Colab.

2. Project Objectives

- Understand the structure and quality of the retail dataset.
- Clean and prepare the data for analysis.
- Calculate total sales, total profit, order count, customer count and profit margin.
- Compare category-wise and region-wise sales and profit.
- Analyze monthly sales trends.
- Find the top 10 products by sales and profit.
- Save a cleaned dataset for later SQL and dashboard work.

3. Tools Used

Python, Pandas and Matplotlib, running in Google Colab. Pandas is used for data handling and calculations. Matplotlib is used to create charts.

4. Step-by-Step Python Analysis

Step 1 – Import the required libraries

First import Pandas for data analysis and Matplotlib for charts. Run this cell before using the libraries.

```
import pandas as pd
import matplotlib.pyplot as plt
print('Libraries imported successfully')
```

Expected output: Libraries imported successfully

Step 2 – Upload the dataset to Google Colab

Upload the Sample Superstore CSV file from your computer. The filename used below should match the uploaded file exactly. If your file is already in the Colab session, skip the upload command and use its exact filename.

```
from google.colab import files
uploaded = files.upload()

# After upload, set the exact filename shown by Colab
filename = 'Sample - Superstore.csv'
```

Expected output: Colab displays the uploaded filename. Make sure filename matches it exactly.

Step 3 – Load the dataset

Read the CSV file into a DataFrame named df. The latin1 encoding was used in this project to read the dataset successfully.

```
df = pd.read_csv(filename, encoding='latin1')

print('Dataset loaded successfully')
```

Expected output: Dataset loaded successfully

Step 4 – View the first five rows

Use head() to preview the first five records and understand what each row contains.

```
df.head()
```

Expected output: A table showing the first five records, including fields such as Order Date, Sales, Profit, Region, Category and Product Name.

Step 5 – Check dataset size

shape returns the number of rows and columns. The original dataset contains 9,994 rows and 21 columns.

```
print('Dataset Shape:', df.shape)
```

Expected output: Dataset Shape: (9994, 21)

Step 6 – Check column names and data types

info() shows column names, non-null counts and data types. This helps identify date columns and fields that may need conversion.

```
df.info()
```

Expected output: The dataset information shows 9,994 entries and 21 columns. Date fields may initially be read as object/string values.

Step 7 – Check missing values

Count missing values in every column. This confirms whether any columns contain blank values.

```
print(df.isnull().sum())
```

Expected output: A list of column names with their missing-value counts. Use the actual output from your Colab notebook in the final submission if it differs.

Step 8 – Check duplicate rows

Duplicate rows can affect totals. Count exact duplicate records before deciding whether any need to be removed.

```
print('Duplicate Rows:', df.duplicated().sum())
```

Expected output: A single number showing the number of duplicate rows found.

Step 9 – Convert date columns

Convert Order Date and Ship Date into Pandas datetime values. This makes date-based analysis possible.

```
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])

print(df[['Order Date', 'Ship Date']].dtypes)
```

Expected output: Both Order Date and Ship Date display datetime64[ns].

Step 10 – Create the Order_Month column

Create a month field from Order Date. It is used to group sales and profit by month.

```
df['Order_Month'] = df['Order Date'].dt.to_period('M').astype(str)

print(df[['Order Date', 'Order_Month']].head())
```

Expected output: A new Order_Month column appears with values formatted like 2014-01, 2014-02, and so on.

Step 11 – Review the cleaned dataset

Check the shape after adding Order_Month. The row count remains 9,994 and the new column increases the number of columns to 22.

```
print('Cleaned Dataset Shape:', df.shape)
df.head()
```

Expected output: Cleaned Dataset Shape: (9994, 22)

Step 12 – Calculate total sales and total profit

sum() adds the values in the Sales and Profit columns to calculate overall business performance.

```
total_sales = df['Sales'].sum()
total_profit = df['Profit'].sum()

print('Total Sales: ${:,.2f}'.format(total_sales))
print('Total Profit: ${:,.2f}'.format(total_profit))
```

Expected output: Total Sales: $2,297,200.86
Total Profit: $286,397.02

Step 13 – Calculate total orders and customers

nunique() counts distinct order IDs and customer IDs. This avoids counting the same order or customer repeatedly.

```
total_orders = df['Order ID'].nunique()
total_customers = df['Customer ID'].nunique()

print('Total Orders:', total_orders)
print('Total Customers:', total_customers)
```

Expected output: Total Orders: 5009
Total Customers: 793

Step 14 – Calculate profit margin

Profit margin is the percentage of sales retained as profit. The formula is total profit divided by total sales, multiplied by 100.

```
profit_margin = (total_profit / total_sales) * 100
print('Profit Margin: {:.2f}%'.format(profit_margin))
```

Expected output: Profit Margin: 12.47%

Step 15 – Sales and profit by category

groupby() groups rows by Category. agg() calculates total sales and total profit for each category.

```
category_analysis = df.groupby('Category').agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum')
).reset_index()

print(category_analysis)
```

Expected output: A three-row summary for Furniture, Office Supplies and Technology, with Total_Sales and Total_Profit columns. Technology is the highest-performing category by sales and profit.

Step 16 – Sales and profit by region

Group the data by Region to compare the four regions.

```
region_analysis = df.groupby('Region').agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum')
).reset_index()

print(region_analysis)
```

Expected output: A four-row summary for Central, East, South and West. West has the highest sales.

Step 17 – Monthly sales and profit

Group records by Order_Month. Sorting by month keeps the timeline in chronological order.

```
monthly_analysis = df.groupby('Order_Month').agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum')
).reset_index().sort_values('Order_Month')

print(monthly_analysis.head())
```

Expected output: A month-by-month table with Total_Sales and Total_Profit. The highest sales month in the completed analysis was November 2017 ($118,447.82); the lowest was February 2014 ($4,519.89).

Step 18 – Top 10 products by sales

Group by Product Name, calculate total sales and sort from highest to lowest. head(10) returns the first ten products.

```
top_sales = df.groupby('Product Name')['Sales'].sum().sort_values(ascending=False).head(10)
print(top_sales)
```

Expected output: The top product is Canon imageCLASS 2200 Advanced Copier, with sales of $61,599.82.

Step 19 – Top 10 products by profit

Repeat the ranking using the Profit column to identify the products generating the highest total profit.

```
top_profit = df.groupby('Product Name')['Profit'].sum().sort_values(ascending=False).head(10)
print(top_profit)
```

Expected output: The top product is Canon imageCLASS 2200 Advanced Copier, with profit of $25,199.93.

Step 20 – Create a category sales chart

A bar chart makes category sales easier to compare visually.

```
category_sales = df.groupby('Category')['Sales'].sum()
category_sales.plot(kind='bar', title='Sales by Category')
plt.xlabel('Category')
plt.ylabel('Sales')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

Expected output: A bar chart with three bars. Technology has the tallest bar.

Step 21 – Create a regional sales chart

This chart compares total sales across the four regions.

```
region_sales = df.groupby('Region')['Sales'].sum()
region_sales.plot(kind='bar', title='Sales by Region')
plt.xlabel('Region')
plt.ylabel('Sales')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

Expected output: A four-bar chart. West has the tallest bar.

Step 22 – Create a monthly sales trend chart

A line chart shows how sales change from month to month.

```
monthly_sales = df.groupby('Order_Month')['Sales'].sum().sort_index()
plt.figure(figsize=(14, 5))
monthly_sales.plot(kind='line', marker='o')
plt.title('Monthly Sales Trend')
plt.xlabel('Order Month')
plt.ylabel('Sales')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

Expected output: A line chart showing monthly sales across the dataset period.

Step 23 – Export the cleaned dataset

Save the prepared dataset so it can be reused in the SQL notebook and dashboard without repeating the cleaning steps.

```
df.to_csv('Retail_Cleaned_Data.csv', index=False)

from google.colab import files
files.download('Retail_Cleaned_Data.csv')
```

Expected output: Colab downloads a file named Retail_Cleaned_Data.csv. The cleaned file contains 9,994 rows and 22 columns.

5. Consolidated Results

Metric	Result
Rows in original dataset	9,994
Original columns	21
Cleaned columns	22
Total Sales	$2,297,200.86
Total Profit	$286,397.02
Total Orders	5,009
Total Customers	793
Profit Margin	12.47%
Top product by sales	Canon imageCLASS 2200 Advanced Copier
Top product by profit	Canon imageCLASS 2200 Advanced Copier

6. Conclusion

The Python analysis transformed the raw Sample Superstore data into a cleaned dataset and a set of useful business summaries. It measured overall performance, compared categories and regions, identified monthly trends and ranked products. The cleaned CSV was then available for the separate SQL analysis and interactive dashboard.

7. Important Notes

The code blocks in this report are clean reproducible examples based on the project workflow. Run them in order in Google Colab. If the runtime disconnects, rerun the import, upload and data-loading cells before continuing because variables are cleared when a Colab runtime restarts.

Expected outputs are summarized from the project results available in the conversation. For a formal academic submission, replace any general output descriptions (such as missing-value and duplicate counts) with the exact output copied from your own notebook.

24. Outputs

Figure 1. Monthly Sales Trend

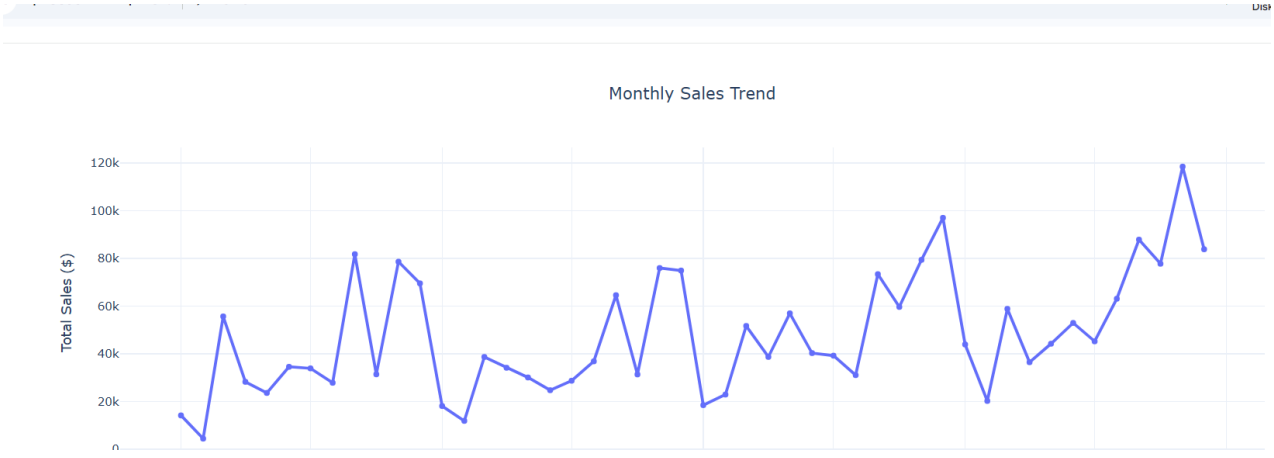


Figure 2. Sales by Product Category



Figure 3. Sales by Region

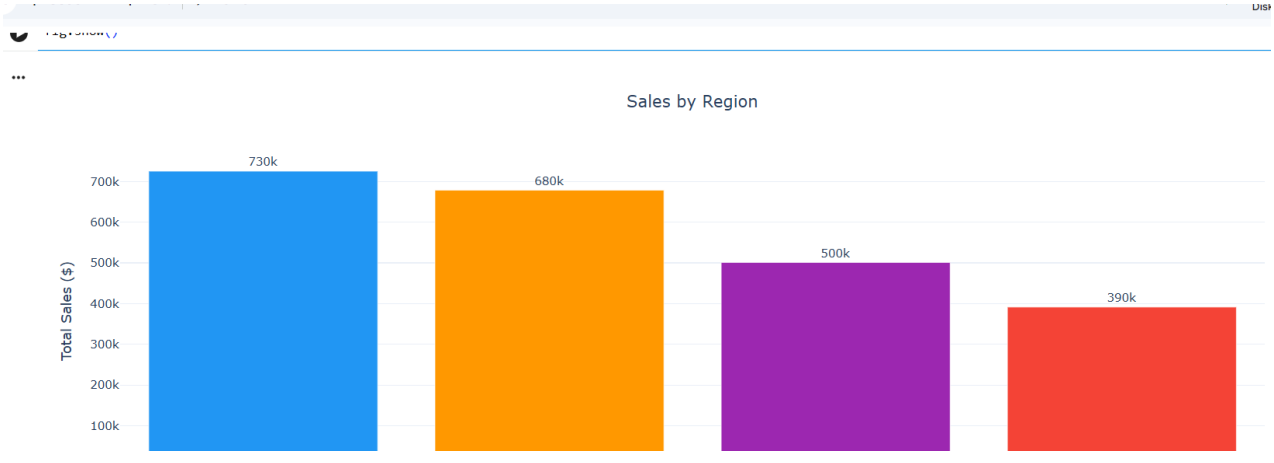


Figure 4. Sales by Region and Top 10 Products by Sales

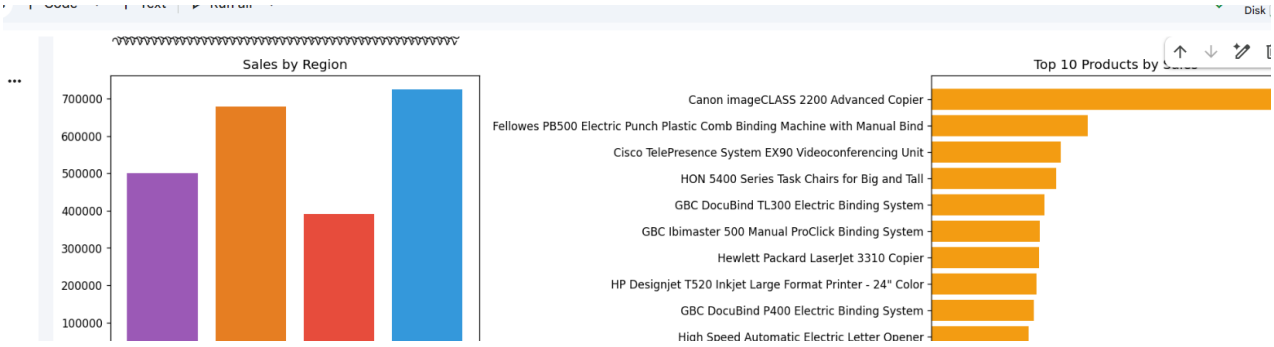


Figure 5. Top 10 Products by Profit

